COMPLETE TECHNICAL DOCUMENTATION

Frieren's Reactive Combat System

Complete Architecture, Lore Mechanics, and C# Source Code Reference

Project Name	FrierenCombat (Frieren-Game)
Target Framework	.NET 10.0 (net10.0)
Language Version	C# 13 (LangVersion: latest)
SDK & Targeting Pack	SDK 10.0.113 / Microsoft.NETCore.App 10.0.13
Target Audience	Programming Instructors & Curriculum Developers
Architecture Pattern	Object-Oriented, Event-Driven Domain Modeling
External Dependencies	None (0 NuGet packages; pure Base Class Library)

Authored for educational curricula and technical review • Generated from current repository source files

1. Architectural Overview & System Summary

Frieren's Reactive Combat System is an object-oriented, event-driven tactical battle simulation written in C# on .NET 10. The project faithfully translates the magic and perception principles of *Frieren: Beyond Journey's End (Sousou no Frieren)* into software engineering paradigms.

> **Core Educational Objective**
>
> This project is designed as an exemplary instructional codebase demonstrating: **Inheritance & Polymorphism**, **Encapsulation with Invariants**, **Multicast Event-Driven Decoupling**, and **Rule-Based AI Decision Trees** in modern C#.

The system is comprised of three primary domains:

1. **The Combatant Hierarchy:** A base `Character` class providing hit points, event dispatching, and damage reduction mechanics, subclassed into `Mage` (mana, spells, suppression) and `Demon` (innate magic, pride, perception).
2. **Lore-Accurate Combat Mechanics:** Implementation of mana suppression deception (Flamme's technique) where demonic opponents size up threats strictly by perceived mana, exposing guard vulnerabilities when true mana is unveiled.
3. **Application Driver & Interactive CLI:** A robust console engine offering interactive turn-based scenario duels, ASCII dynamic resource bars, and automated headless demonstration execution.

2. Project Structure & File Map

The solution is organized cleanly as a single executable project without superfluous nesting:

File Path	Type / Responsibility	Role in Application
FrierenCombat.csproj	Project Configuration	Defines SDK configuration, output type (executable), .NET 10 target, implicit usings, and nullable reference checks.
.gitignore	Version Control Filter	Excludes build artifacts (`bin/` , `obj/`) and IDE metadata (`.vs/` , `.vscode/`) from Git.
Character.cs	Root Domain Model	Base combatant encapsulating Health, MaxHealth, Block, Heal, Attack, TakeDamage, and lifecycle events.
Mage.cs	Specialized Entity	Human/Elf magic-user adding Mana, Zoltraak (demon-slaying magic), Defensive Barriers, and Mana Suppression.
Demon.cs	Specialized Entity	Antagonist subclass adding Innate Magic, Demonic Pride, atmospheric mana regeneration, and opponent evaluation.
Program.cs	Application Orchestrator	Contains the entry point (`Main`), interactive scenario menu, AI decision engine, HUD rendering, and demo runner.

3. Detailed Class & Member Specifications

3.1 Character (Base Class)

Location: `Character.cs` • **Accessibility:** `public`

Member	Type	Access	Description
`Name`	Property (`string`)	`public get; private set;`	Trimmed, validated name of the character.
`Health`	Property (`int`)	`public get; protected set;`	Current hit points, bounded between 0 and `MaxHealth`.
`MaxHealth`	Property (`int`)	`public get; private set;`	Maximum capacity of hit points. Must be > 0.
`IsBlocking`	Property (`bool`)	`public get; private set;`	Defensive stance flag. When true, halves next incoming damage.
`IsAlive`	Property (`bool`)	`public get;`	Expression-bodied computed property: `Health > 0`.
`Damaged`	Event (`Action<Character, int>?`)	`public`	Fired when character loses health. Passes sender and damage taken.
`Healed`	Event (`Action<Character, int>?`)	`public`	Fired when character is healed. Passes sender and points restored.
`Defeated`	Event (`Action<Character>?`)	`public`	Fired when health reaches 0. Passes sender.
`Character(...)`	Constructor	`public`	Validates arguments and initializes HP and blocking state.
`Block()`	Virtual Method (`void`)	`public`	Enters defensive guard if alive and not currently blocking.
`ResetBlock()`	Method (`void`)	`public`	Resets `IsBlocking = false`.
`TakeDamage(int)`	Virtual Method (`int`)	`public`	Applies blocking mitigation, subtracts HP, triggers events, and returns damage dealt.
`Damage(int)`	Virtual Method (`void`)	`public`	Backward compatibility alias pointing to `TakeDamage(amount)`.
`Heal(int)`	Virtual Method (`int`)	`public`	Restores health up to MaxHealth and triggers `Healed` event.
`Attack(...)`	Virtual Method (`void`)	`public`	Dispatches an attack of specified damage (default 10) to a target.
`OnDamaged(int)`	Virtual Method (`void`)	`protected`	Invokes `Damaged` event; prints critical injury warning if HP ≤ 25%.
`OnHealed(int)`	Virtual Method (`void`)	`protected`	Invokes `Healed` event; prints full vitality message if HP reaches Max.
`OnDeath()`	Virtual Method (`void`)	`protected`	Clears blocking stance, prints defeat message, and invokes `Defeated`.

3.2 Mage (Subclass)

Location: `Mage.cs` • **Accessibility:** `public` • **Base:** `Character`

Member	Type	Access	Description
`Mana`	Property (`int`)	`public get; protected set;`	Current mana reserve available for spellcasting.
`MaxMana`	Property (`int`)	`public get; protected set;`	Maximum capacity of mana. Must be > 0.
`IsSuppressingMana`	Property (`bool`)	`public get; private set;`	True if mana flow is intentionally concealed.
`HasDefensiveBarrier`	Property (`bool`)	`public get; private set;`	True if a hexagonal shield is active to absorb 75% damage.
`PerceivedMana`	Property (`int`)	`public get;`	Visible mana: returns `Max(1, MaxMana / 10)` when suppressed, else `Mana`.
`SpellCast`	Event (`Action<Mage, string>?`)	`public`	Fired whenever a spell is cast. Passes mage and spell name.
`ManaSuppressionChanged`	Event (`Action<Mage, bool>?`)	`public`	Fired when suppression toggles. Passes mage and new state.
`ToggleManaSuppression()`	Method (`void`)	`public`	Toggles suppression state, logs atmosphere text, fires event.
`SetManaSuppression(bool)`	Method (`void`)	`public`	Sets suppression to explicit boolean value if different.
`CastBasicSpell(Character)`	Virtual Method (`bool`)	`public`	Costs 10 MP, deals 20 damage. Fires `SpellCast`.
`CastZoltraak(Character)`	Virtual Method (`bool`)	`public`	Costs 25 MP, deals 45 dmg (67 dmg vs Demon). Fires `SpellCast`.
`DeployDefensiveBarrier()`	Virtual Method (`bool`)	`public`	Costs 15 MP. Activates 75% damage absorption barrier.
`Meditate(int)`	Virtual Method (`int`)	`public`	Restores specified MP (default 30) up to MaxMana.
`TakeDamage(int)`	Override Method (`int`)	`public`	Absorbs 75% if barrier active, consumes barrier, forwards residual to base.

3.3 Demon (Subclass)

Location: `Demon.cs` • **Accessibility:** `public` • **Base:** `Character`

Member	Type	Access	Description
`Mana`	Property (`int`)	`public get; protected set;`	Current demonic mana reserve.
`MaxMana`	Property (`int`)	`public get; protected set;`	Upper limit of demonic mana.
`InnateMagicName`	Property (`string`)	`public get; init;`	Name of unique innate magic (e.g. Auserlese, Baelsewer).
`Title`	Property (`string`)	`public get; init;`	Demonic rank / title (e.g. Seven Sages of Destruction).
`PrideLevel`	Property (`int`)	`public get; protected set;`	Demonic pride metric (fixed at 100).
`InnateMagicCast`	Event (`Action<Demon, string>?`)	`public`	Fired when innate magic is cast. Passes demon and spell name.
`UnderestimatesOpponent(...)`	Method (`bool`)	`public`	Returns true if opponent is a Mage whose `PerceivedMana < Mana / 2`.
`CastInnateMagic(Character)`	Virtual Method (`bool`)	`public`	Costs 30 MP, deals 38 damage. Fallback to physical attack if MP < 30.
`CastSpell(Character)`	Virtual Method (`void`)	`public`	Costs 15 MP, deals 24 damage dark mana blast.
`TakeDamageFrom(...)`	Virtual Method (`int`)	`public`	Applies +35% damage penalty if demon underestimated an unsuppressed mage.
`RegenerateMana(int)`	Virtual Method (`int`)	`public`	Draws atmospheric mana (default +20 MP) up to MaxMana.
`OnDeath()`	Override Method (`void`)	`protected`	Calls base death method and outputs mana particle disintegration text.

3.4 Program (Orchestrator)

Location: `Program.cs` • **Accessibility:** `internal static`

Method	Return	Description
`Main(string[] args)`	`void`	Entry point. Evaluates flags (`--demo` , `--test` , `-d`) and stdin redirection; dispatches to demo or interactive menu.
`RunMainMenu()`	`void`	Displays interactive scenario selection loop and dispatches user choices.
`RunInteractiveBattle(...)`	`void`	Coordinates turn-by-turn combat between player Mage and Demon boss until defeat.
`ExecuteDemonTurn(...)`	`void`	Implements demonic AI decision tree based on MP and HP thresholds.
`RunAutomatedDemonstration()`	`void`	Runs scripted, 4-round demonstration of Frieren vs Aura showcasing all mechanics.
`DisplayCombatantHUD(...)`	`void`	Prints visual status bars for HP, MP, barriers, and suppression tags.
`GenerateBar(...)`	`string`	Constructs filled (`▉`) and unfilled (`▒`) Unicode bar strings.
`PrintBanner()`	`void`	Outputs decorative ASCII title banner.

4. Class Relationships & Inheritance Model

The project utilizes single inheritance and polymorphic interface design. Both `Mage` and `Demon` specialize the abstract concept of a `Character` while adding asymmetric capabilities.

1. **Character (Base Abstraction)**
   Encapsulates fundamental life attributes: Hit Points, physical blocking stance, damage resolution, healing, and lifecycle events (Damaged, Healed, Defeated).
2. **Mage ← Inherits Character**
   Extends Character with human/elven magic attributes: Mana pool, Zoltraak offensive spell, hexagonal defensive barrier, meditation, and mana suppression state.
3. **Demon ← Inherits Character**
   Extends Character with demonic attributes: Innate unique magic, atmospheric mana regeneration, demonic pride, and opponent threat evaluation logic.
4. **Program • Composition & Orchestration**
   Instantiates concrete Mage and Demon instances, wires event listeners to console output, and mediates interactive turns and AI actions.

5. Execution Flow & Lifecycle

The application follows a structured lifecycle starting from `Program.Main`:

1. **Application Bootstrap (Program.Main)**
   Configures `Console.OutputEncoding = Encoding.UTF8` for Unicode rendering. Inspects CLI arguments and input redirection. If headless/testing, runs `RunAutomatedDemonstration()` and exits; otherwise runs `RunMainMenu()`.
2. **Main Menu Selection (RunMainMenu)**
   Clears console, displays ASCII banner, and presents 5 options. Options 1-3 construct tailored Mage and Demon instances and initiate `RunInteractiveBattle`.
3. **Combat Loop Initialization (RunInteractiveBattle)**
   Applies initial suppression if configured. Attaches event handlers to `player.Defeated` and `boss.Defeated`. Starts turn counter at 1.
4. **Turn Execution Cycle**
   A. HUD Display: Renders visual HP/MP bars for player and boss.
   B. Player Input: Prompts action (Zoltraak, Barrier, Suppression, Basic Magic, Staff Strike, Meditate, Block).
   C. Life Check: Breaks immediately if the Demon boss has fallen.
   D. Demon AI Turn: `ExecuteDemonTurn` evaluates mana and health thresholds to execute innate magic, spell, block, or attack.
   E. Demon Passive Regen: Demon regenerates +10 MP at end of turn.
   F. Advance: Increments turn counter and prompts user to advance.
5. **Combat Resolution**
   Outputs formal VICTORY or DEFEAT banner, waits for user confirmation, and returns cleanly to the main menu.

6. Comprehensive Method Call Map

The table below documents cross-file and intra-class call paths, parameter types, and invocation semantics:

Caller	Callee	Location	Data Passed / Semantics
`Program.Main`	`RunAutomatedDemonstration`	`Program.cs`	Invoked if `--demo`, `--test`, or input redirected.
`Program.Main`	`RunMainMenu`	`Program.cs`	Invoked for standard interactive console sessions.
`Program.RunMainMenu`	`RunInteractiveBattle`	`Program.cs`	Passes configured `Mage`, `Demon`, initial suppression flag, and lore text.
`RunInteractiveBattle`	`Mage.SetManaSuppression`	`Mage.cs`	Passes `true` for Frieren scenario setup.
`RunInteractiveBattle`	`Mage.CastZoltraak`	`Mage.cs`	Passes Demon target. Deducts 25 MP; deals 67 demon-slaying damage.
`RunInteractiveBattle`	`Mage.DeployDefensiveBarrier`	`Mage.cs`	Deducts 15 MP; sets `HasDefensiveBarrier = true`.
`RunInteractiveBattle`	`Mage.ToggleManaSuppression`	`Mage.cs`	Flips suppression state; fires `ManaSuppressionChanged`.
`RunInteractiveBattle`	`ExecuteDemonTurn`	`Program.cs`	Passes Demon and Mage to trigger AI decision tree.
`ExecuteDemonTurn`	`Demon.CastInnateMagic`	`Demon.cs`	Passes player Mage target; consumes 30 MP; deals 38 damage.
`ExecuteDemonTurn`	`Demon.CastSpell`	`Demon.cs`	Passes player Mage target; consumes 15 MP; deals 24 damage.
`Mage.CastZoltraak`	`Demon.TakeDamageFrom`	`Demon.cs`	Passes `this` (Mage) and calculated damage. Checks deception multiplier.
`Demon.TakeDamageFrom`	`Character.TakeDamage`	`Character.cs`	Applies 1.35x multiplier if guard broken, then calls base damage method.
`Mage.TakeDamage`	`base.TakeDamage`	`Character.cs`	Absorbs 75% if barrier active, then passes residual 25% to base.
`Character.TakeDamage`	`Character.OnDamaged`	`Character.cs`	Fires `Damaged` event; checks critical HP alert.
`Character.TakeDamage`	`Character.OnDeath`	`Character.cs`	Fires `Defeated` event if HP drops to 0.

7. Combat Mechanics & Mathematical Formulas

7.1 Damage Mitigation Pipeline

Incoming damage is resolved through three deterministic layers:

1. **Demon Guard Break Multiplier:** If target is a Demon who underestimated an opponent who is now unsuppressed, damage is increased by 35%:

   > **Formula:** `FinalDamage = floor(Amount * 1.35)`

2. **Hexagonal Defensive Barrier (Mage):** If `HasDefensiveBarrier == true`, absorbs 75% of incoming damage and is consumed:

   > **Formula:** `Absorbed = floor(Damage * 0.75)` • `Residual = Damage - Absorbed`

3. **Physical Stance Block (Character):** If `IsBlocking == true`, incoming damage is halved and blocking is consumed:

   > **Formula:** `ActualDamage = floor(Residual / 2)`

7.2 Lore-Accurate Mana Deception

> **Flamme's Principle of Mana Concealment**
>
> In *Frieren*, demons believe mana represents supreme status and absolute strength. They consider mana concealment unthinkable and gauge opponents solely by visible mana aura.

- **Suppressed Perceived Mana:** When `IsSuppressingMana == true`, apparent mana is: `PerceivedMana = Max(1, MaxMana / 10)` (e.g. Frieren with 200 MaxMana appears to have only 20 MP).
- **Demonic Evaluation:** The demon underestimates an opponent if: `opponent.PerceivedMana < (demon.Mana / 2)`.
- **Guard Break:** When the mage unleashes their true mana aura, the demon is shocked by the disparity, leaving their defense exposed to the 35% damage penalty.

8. Events & Reactive Architecture

All lifecycle and combat occurrences are surfaced via standard C# events utilizing `System.Action<...>` delegates:

Class	Event Name	Delegate Signature	Trigger Condition	Subscriber Purpose
`Character`	`Damaged`	`Action<Character, int>?`	After HP reduction in `TakeDamage`.	UI logging, damage notification, audio triggers.
`Character`	`Healed`	`Action<Character, int>?`	After HP increase in `Heal`.	UI restoration alerts.
`Character`	`Defeated`	`Action<Character>?`	When HP reaches 0 in `OnDeath`.	Subscribed by `Program` to display combat death announcements.
`Mage`	`SpellCast`	`Action<Mage, string>?`	Upon casting Basic Magic, Zoltraak, or Barrier.	Spell animation triggers, telemetry logging.
`Mage`	`ManaSuppressionChanged`	`Action<Mage, bool>?`	When suppression is toggled.	Aura visual effect shifts, AI awareness recalculation.
`Demon`	`InnateMagicCast`	`Action<Demon, string>?`	Upon casting innate unique demonic spell.	Boss visual effects, cutscene dialogue.

9. Configuration, Runtime & Dependencies

9.1 Project Settings (FrierenCombat.csproj)

- **SDK:** `Microsoft.NET.Sdk`
- **Output Type:** `Exe` (Standalone Console Application)
- **Target Framework:** `net10.0` (.NET 10.0 runtime)
- **Language Version:** `latest` (C# 13)
- **Implicit Usings:** `enable` (Auto-imports System, System.Collections.Generic, System.IO, System.Linq, System.Threading)
- **Nullable:** `enable` (Static compile-time null safety enforcement)

9.2 Dependencies

- **Third-Party Packages: 0 NuGet dependencies**. The project is completely self-contained and compiles purely with the .NET 10 Base Class Library.
- **System Libraries:** `System`, `System.Text`.
- **Cross-Platform Compatibility:** Runs natively on Linux, macOS, and Windows with any terminal supporting UTF-8.

10. Complete, Unmodified Source Code

The following subsections provide the complete, unmodified source code for all six repository files, exactly as preserved on disk.

10.1 FrierenCombat.csproj

FrierenCombat.csproj — MSBuild / XML • 11 Lines

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <LangVersion>latest</LangVersion>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

10.2 .gitignore

.gitignore — Git Configuration • 11 Lines

```
# Build results
[Bb]in/
[Oo]bj/

# Visual Studio / Rider / VS Code cache
.vs/
.idea/
.vscode/
*.user
*.suo

```

10.3 Character.cs

Character.cs — C# • 178 Lines

```

public class Character
{
    public string Name { get; private set; }
    public int Health { get; protected set; }
    public int MaxHealth { get; private set; }
    public bool IsBlocking { get; private set; }
    public bool IsAlive => Health > 0;

    // Combat lifecycle events
    public event Action<Character, int>? Damaged;
```

```
    public event Action<Character, int>? Healed;
    public event Action<Character>? Defeated;

    public Character(string name, int maxHealth)
    {
        if (string.IsNullOrWhiteSpace(name))
            throw new ArgumentException("Character name cannot be null or empty.", nameof(name));

        if (maxHealth <= 0)
            throw new ArgumentOutOfRangeException(nameof(maxHealth), "Max health must be greater tha
n 0.");

        Name = name.Trim();
        MaxHealth = maxHealth;
        Health = maxHealth;
        IsBlocking = false;
    }

    public virtual void Block()
    {
        if (!IsAlive)
        {
            Console.WriteLine($"{Name} cannot block because they are defeated.");
            return;
        }

        if (IsBlocking)
        {
            Console.WriteLine($"{Name} is already in a defensive stance.");
            return;
        }

        IsBlocking = true;
        Console.WriteLine($"{Name} prepares to block incoming attacks!");
    }

    public void ResetBlock()
    {
        IsBlocking = false;
    }

    public virtual int TakeDamage(int amount)
    {
        if (!IsAlive)
        {
            Console.WriteLine($"{Name} is already defeated.");
            return 0;
        }

        if (amount <= 0)
        {
            Console.WriteLine($"{Name} was targeted, but took no damage.");
            return 0;
        }

        int actualDamage = amount;

        if (IsBlocking)
```

```
        {
            actualDamage = amount / 2;
            IsBlocking = false;
            Console.WriteLine($"{Name} blocks half the damage!");
        }

        int previousHealth = Health;
        Health = Math.Max(0, Health - actualDamage);
        int damageDealt = previousHealth - Health;

        Console.WriteLine($"{Name} takes {damageDealt} damage. HP: {Health}/{MaxHealth}");

        OnDamaged(damageDealt);

        if (!IsAlive)
        {
            OnDeath();
        }

        return damageDealt;
    }

    // Retained for backward compatibility with existing code calling Damage(amount)
    public virtual void Damage(int amount) => TakeDamage(amount);

    public virtual int Heal(int amount)
    {
        if (!IsAlive)
        {
            Console.WriteLine($"{Name} is defeated and cannot be healed.");
            return 0;
        }

        if (amount <= 0)
            return 0;

        int previousHealth = Health;
        Health = Math.Min(MaxHealth, Health + amount);
        int healedAmount = Health - previousHealth;

        if (healedAmount > 0)
        {
            Console.WriteLine($"{Name} heals for {healedAmount} HP. HP: {Health}/{MaxHealth}");
            OnHealed(healedAmount);
        }
        else
        {
            Console.WriteLine($"{Name} is already at full health (HP: {Health}/{MaxHealth}).");
        }

        return healedAmount;
    }

    public virtual void Attack(Character target, int damage = 10)
    {
        ArgumentNullException.ThrowIfNull(target);

        if (!IsAlive)
```

```
        {
            Console.WriteLine($"{Name} cannot attack because they are defeated.");
            return;
        }

        if (target == this)
        {
            Console.WriteLine($"{Name} cannot attack themselves.");
            return;
        }

        Console.WriteLine($"{Name} attacks {target.Name}!");
        target.TakeDamage(damage);
    }

    protected virtual void OnDamaged(int damageTaken)
    {
        // Trigger external subscribers
        Damaged?.Invoke(this, damageTaken);

        // Warning trigger when HP falls into critical range (<= 25%)
        if (IsAlive && Health <= MaxHealth * 0.25)
        {
            Console.WriteLine($"⚠️ [Critical] {Name} is critically injured! (HP: {Health}/{MaxHealth})");
        }
    }

    protected virtual void OnHealed(int amountHealed)
    {
        // Trigger external subscribers
        Healed?.Invoke(this, amountHealed);

        // Notification if restored to full vitality
        if (Health >= MaxHealth)
        {
            Console.WriteLine($"{Name} has returned to maximum vitality!");
        }
    }

    protected virtual void OnDeath()
    {
        // Clear lingering combat states
        IsBlocking = false;

        Console.WriteLine($"{Name} has fallen in battle!");

        // Trigger external subscribers
        Defeated?.Invoke(this);
    }

    public override string ToString() => $"{Name} (HP: {Health}/{MaxHealth}, Status: {(IsAlive ? "Alive" : "Defeated")})";
}
```

10.4 Mage.cs

Mage.cs C# • 201 Lines

```
public class Mage : Character
{
    public int Mana { get; protected set; }
    public int MaxMana { get; protected set; }
    public bool IsSuppressingMana { get; private set; }
    public bool HasDefensiveBarrier { get; private set; }

    // Mage events
    public event Action<Mage, string>? SpellCast;
    public event Action<Mage, bool>? ManaSuppressionChanged;

    public Mage(string name, int maxHealth) : this(name, maxHealth, 100)
    {
    }

    public Mage(string name, int maxHealth, int maxMana) : base(name, maxHealth)
    {
        if (maxMana <= 0)
            throw new ArgumentOutOfRangeException(nameof(maxMana), "Max mana must be greater than 0.");

        MaxMana = maxMana;
        Mana = maxMana;
        IsSuppressingMana = false;
        HasDefensiveBarrier = false;
    }

    /// <summary>
    /// Perceived mana seen by opponents.
    /// Flamme's teaching: by concealing mana, the opponent fails to estimate the mage's true strength.
    /// </summary>
    public int PerceivedMana => IsSuppressingMana ? Math.Max(1, MaxMana / 10) : Mana;

    /// <summary>
    /// Toggles the suppression of mana aura.
    /// </summary>
    public void ToggleManaSuppression()
    {
        IsSuppressingMana = !IsSuppressingMana;
        if (IsSuppressingMana)
        {
            Console.WriteLine($"🤫 {Name} flawlessly restricts their mana flow. To outside eyes, their mana appears tiny ({PerceivedMana} MP)!");
        }
        else
        {
            Console.WriteLine($"🌊 {Name} releases their mana concealment! A colossal, breathtaking torrent of mana erupts ({Mana}/{MaxMana} MP)!");
        }
        ManaSuppressionChanged?.Invoke(this, IsSuppressingMana);
    }

    public void SetManaSuppression(bool suppress)
```

```
    {
        if (IsSuppressingMana != suppress)
            ToggleManaSuppression();
    }

    /// <summary>
    /// Basic spell casting method (retained for backward compatibility).
    /// </summary>
    public virtual void CastSpell(Character target)
    {
        CastBasicSpell(target);
    }

    /// <summary>
    /// Standard magical projectile attack.
    /// </summary>
    public virtual bool CastBasicSpell(Character target)
    {
        ArgumentNullException.ThrowIfNull(target);

        const int manaCost = 10;
        const int damage = 20;

        if (!IsAlive)
        {
            Console.WriteLine($"{Name} cannot cast spells because they are defeated.");
            return false;
        }

        if (Mana < manaCost)
        {
            Console.WriteLine($"{Name} does not have enough mana to cast basic magic! ({Mana}/{manaC
ost} required)");
            return false;
        }

        Mana -= manaCost;
        Console.WriteLine($"🪄 {Name} casts a spell on {target.Name}!");
        SpellCast?.Invoke(this, "Basic Magic");
        target.TakeDamage(damage);
        return true;
    }

    /// <summary>
    /// Zoltraak (Ordinary Offensive Magic) - Humanity's refined demon-piercing offensive spell.
    /// Deals devastating damage, especially against Demons!
    /// </summary>
    public virtual bool CastZoltraak(Character target)
    {
        ArgumentNullException.ThrowIfNull(target);

        const int manaCost = 25;
        int baseDamage = 45;

        if (!IsAlive)
        {
            Console.WriteLine($"{Name} cannot cast Zoltraak because they are defeated.");
            return false;
```

```
        }

        if (Mana < manaCost)
        {
            Console.WriteLine($"{Name} lacks the mana for Zoltraak! ({Mana}/{manaCost} required)");
            return false;
        }

        Mana -= manaCost;

        bool isDemon = target is Demon;
        int actualDamage = isDemon ? (int)(baseDamage * 1.5) : baseDamage;

        Console.WriteLine($"✨ {Name} casts [Zoltraak - Ordinary Offensive Magic] targeting {target.Name}!");
        if (isDemon)
        {
            Console.WriteLine($"   ⚡ The piercing white beam tears through demonic defenses (+50% demon-slaying bonus)!");
        }

        SpellCast?.Invoke(this, "Zoltraak");

        if (target is Demon demon)
        {
            demon.TakeDamageFrom(this, actualDamage);
        }
        else
        {
            target.TakeDamage(actualDamage);
        }

        return true;
    }

    /// <summary>
    /// Deploys a hexagonal defensive barrier that heavily absorbs the next incoming strike.
    /// </summary>
    public virtual bool DeployDefensiveBarrier()
    {
        const int manaCost = 15;

        if (!IsAlive)
        {
            Console.WriteLine($"{Name} cannot deploy a barrier while defeated.");
            return false;
        }

        if (Mana < manaCost)
        {
            Console.WriteLine($"{Name} does not have enough mana for a Defensive Barrier! ({Mana}/{manaCost} required)");
            return false;
        }

        Mana -= manaCost;
        HasDefensiveBarrier = true;
```

```
        Console.WriteLine($"🛡️ {Name} deploys a hexagonal [Defensive Barrier]! Incoming attack damage will be heavily absorbed.");
        SpellCast?.Invoke(this, "Defensive Barrier");
        return true;
    }

    /// <summary>
    /// Meditates to gather natural mana.
    /// </summary>
    public virtual int Meditate(int amount = 30)
    {
        if (!IsAlive)
        {
            Console.WriteLine($"{Name} cannot meditate while defeated.");
            return 0;
        }

        int previousMana = Mana;
        Mana = Math.Min(MaxMana, Mana + amount);
        int recovered = Mana - previousMana;
        Console.WriteLine($"🧘 {Name} calms their mind and focuses mana flow, recovering {recovered} MP ({Mana}/{MaxMana}).");
        return recovered;
    }

    public override int TakeDamage(int amount)
    {
        if (HasDefensiveBarrier && IsAlive && amount > 0)
        {
            int absorbed = (int)(amount * 0.75);
            int residual = amount - absorbed;
            Console.WriteLine($"🛡️ {Name}'s hexagonal barrier absorbs {absorbed} damage, fracturing from the impact!");
            HasDefensiveBarrier = false;
            return base.TakeDamage(residual);
        }

        return base.TakeDamage(amount);
    }

    public override string ToString() =>
        $"{Name} [Mage] (HP: {Health}/{MaxHealth}, MP: {Mana}/{MaxMana}, Perceived MP: {PerceivedMana}, Status: {(IsAlive ? "Alive" : "Defeated")})";
}
```

10.5 Demon.cs

Demon.cs — C# • 138 Lines

```
public class Demon : Character
{
    public int Mana { get; protected set; }
    public int MaxMana { get; protected set; }
    public string InnateMagicName { get; init; }
    public string Title { get; init; }
    public int PrideLevel { get; protected set; } // Pride in their vast mana reserves

    // Demonic event hooks
```

```
    public event Action<Demon, string>? InnateMagicCast;

    public Demon(string name, int maxHealth, int maxMana = 120, string innateMagicName = "Original Zoltraak", string title = "Demon")
        : base(name, maxHealth)
    {
        if (maxMana <= 0)
            throw new ArgumentOutOfRangeException(nameof(maxMana), "Max mana must be greater than 0.");

        MaxMana = maxMana;
        Mana = maxMana;
        InnateMagicName = innateMagicName;
        Title = title;
        PrideLevel = 100;
    }

    /// <summary>
    /// In Frieren lore, demons view mana as their absolute hierarchy, identity, and status.
    /// They cannot conceive of suppressing mana because it defies demonic pride.
    /// If an opponent conceals mana, the demon miscalculates their threat level.
    /// </summary>
    public bool UnderestimatesOpponent(Character opponent)
    {
        if (opponent is Mage mage)
        {
            return mage.PerceivedMana < (Mana / 2);
        }
        return false;
    }

    /// <summary>
    /// Executes the demon's innate signature magic (e.g. Auserlese, Baelsewer, Original Zoltraak).
    /// </summary>
    public virtual bool CastInnateMagic(Character target)
    {
        ArgumentNullException.ThrowIfNull(target);

        const int manaCost = 30;
        const int damage = 38;

        if (!IsAlive)
        {
            Console.WriteLine($"{Name} cannot cast innate magic because they are defeated.");
            return false;
        }

        if (Mana < manaCost)
        {
            Console.WriteLine($"{Name} attempts to summon [{InnateMagicName}], but lacks sufficient mana! ({Mana}/{manaCost} required)");
            Attack(target, 12);
            return false;
        }

        Mana -= manaCost;
        Console.WriteLine($"😈 [{Title}] {Name} activates their terrifying innate magic: [{InnateMagicName}] against {target.Name}!");
```

```
        InnateMagicCast?.Invoke(this, InnateMagicName);

        if (target is Mage mage && UnderestimatesOpponent(mage))
        {
            Console.WriteLine($"   \"{target.Name}... your mana is insignificant. Bow before superior power!\" mocks {Name}.");
        }

        target.TakeDamage(damage);
        return true;
    }

    /// <summary>
    /// Demonic mana blast or ranged assault.
    /// </summary>
    public virtual void CastSpell(Character target)
    {
        ArgumentNullException.ThrowIfNull(target);

        if (!IsAlive)
        {
            Console.WriteLine($"{Name} cannot cast spells because they are defeated.");
            return;
        }

        if (Mana >= 15)
        {
            Mana -= 15;
            Console.WriteLine($"🔮 {Name} releases a wave of dark mana projectiles at {target.Name}!");
            target.TakeDamage(24);
        }
        else
        {
            Attack(target, 12);
        }
    }

    /// <summary>
    /// Deals damage while factoring in whether the demon underestimated a mage who revealed their true mana.
    /// </summary>
    public virtual int TakeDamageFrom(Character attacker, int amount)
    {
        int finalDamage = amount;
        if (attacker is Mage mage && UnderestimatesOpponent(mage) && !mage.IsSuppressingMana)
        {
            Console.WriteLine($"💥 {Name}'s composure shatters upon realizing {attacker.Name}'s disguised mana! Defensive gap exposed!");
            finalDamage = (int)(amount * 1.35);
        }

        return TakeDamage(finalDamage);
    }

    /// <summary>
    /// Passively draws mana from surrounding atmospheric flow.
    /// </summary>
```

```
    public virtual int RegenerateMana(int amount = 20)
    {
        if (!IsAlive) return 0;

        int previousMana = Mana;
        Mana = Math.Min(MaxMana, Mana + amount);
        int recovered = Mana - previousMana;
        Console.WriteLine($"🌀 {Name} gathers demonic mana from the surrounding aura (+{recovered} MP, now {Mana}/{MaxMana}).");
        return recovered;
    }

    protected override void OnDeath()
    {
        base.OnDeath();
        Console.WriteLine($"✨ {Name}'s demonic body disintegrates into glowing mana particles, scattering endlessly into the wind...");
    }

    public override string ToString() =>
        $"{Name} [{Title}] (HP: {Health}/{MaxHealth}, MP: {Mana}/{MaxMana}, Magic: {InnateMagicName}, Status: {(IsAlive ? "Alive" : "Defeated")})";
}

```

10.6 Program.cs

Program.cs C# • 309 Lines

```
using System;
using System.Text;

namespace FrierenCombat;

internal static class Program
{
    private static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;

        // If run with arguments or in non-interactive / redirected mode, execute the automated demo
        if (args.Length > 0 && (args[0] == "--demo" || args[0] == "--test" || args[0] == "-d") || Console.IsInputRedirected)
        {
            RunAutomatedDemonstration();
            return;
        }

        RunMainMenu();
    }

    private static void RunMainMenu()
    {
        while (true)
        {
            Console.Clear();
            PrintBanner();
            Console.WriteLine(" Select a combat scenario:");
```

```
            Console.WriteLine(" [1] ⚔️  Interactive Duel: Frieren vs Aura the Guillotine");
            Console.WriteLine(" [2] 🪄  Interactive Duel: Fern vs Lugner");
            Console.WriteLine(" [3] ⚡  Interactive Boss Duel: Frieren vs Qual (Elder Sage of Corruption)");
            Console.WriteLine(" [4] 🎬  Cinematic Automated Battle Simulation");
            Console.WriteLine(" [5] 🚪  Exit");
            Console.WriteLine();
            Console.Write("Enter your choice (1-5): ");

            string? choice = Console.ReadLine()?.Trim();
            Console.WriteLine();

            switch (choice)
            {
                case "1":
                    RunInteractiveBattle(
                        player: new Mage("Frieren", 120, 200),
                        boss: new Demon("Aura the Guillotine", 110, 160, "Auserlese (Scales of Obedience)", "Great Demon of the Seven Sages of Destruction"),
                        startSuppressed: true,
                        loreIntro: "Aura measures mana on her Scales of Obedience. Unbeknownst to her, Frieren has spent over a millennium mastering mana suppression."
                    );
                    break;
                case "2":
                    RunInteractiveBattle(
                        player: new Mage("Fern", 90, 110),
                        boss: new Demon("Lugner", 95, 100, "Baelsewer (Blood Magic)", "Demon Emissary"),
                        startSuppressed: false,
                        loreIntro: "Lugner relies on malleable blood magic. Fern counters with rapid, relentless casting of Zoltraak."
                    );
                    break;
                case "3":
                    RunInteractiveBattle(
                        player: new Mage("Frieren", 120, 200),
                        boss: new Demon("Qual", 140, 180, "Original Zoltraak (Elder Destruction Magic)", "Elder Sage of Corruption"),
                        startSuppressed: false,
                        loreIntro: "Qual invented Zoltraak 80 years ago. Today, humanity has refined it into ordinary offensive and defensive magic."
                    );
                    break;
                case "4":
                    RunAutomatedDemonstration();
                    break;
                case "5":
                case "q":
                case "exit":
                    Console.WriteLine("Farewell, traveler of the northern lands.");
                    return;
                default:
                    Console.WriteLine("Invalid option. Press any key to return to menu...");
                    Console.ReadKey(true);
                    break;
            }
        }
    }
```

```
81
82      private static void RunInteractiveBattle(Mage player, Demon boss, bool startSuppressed, string l
    oreIntro)
83      {
84          Console.Clear();
85          PrintBanner();
86          Console.WriteLine($"📜 Encounter: {player.Name} vs {boss.Name} ({boss.Title})");
87          Console.WriteLine($"   {loreIntro}");
88          Console.WriteLine();
89
90          if (startSuppressed)
91          {
92              player.SetManaSuppression(true);
93          }
94
95          // Attach event listeners for battle narrations
96          player.Defeated += chara => Console.WriteLine($"💀 [Combat Event] {chara.Name} has succumbed
    to fatal injuries!");
97          boss.Defeated += chara => Console.WriteLine($"🎉 [Combat Event] {chara.Name} has been purged
    from the mortal realm!");
98
99          int turn = 1;
100
101         while (player.IsAlive && boss.IsAlive)
102         {
103             Console.WriteLine(new string('—', 65));
104             Console.WriteLine($"▶ TURN {turn}");
105             Console.WriteLine(new string('—', 65));
106
107             DisplayCombatantHUD(player);
108             DisplayCombatantHUD(boss, player);
109             Console.WriteLine();
110
111             // Player Turn
112             bool actionExecuted = false;
113             while (!actionExecuted)
114             {
115                 Console.WriteLine("Choose your action:");
116                 Console.WriteLine(" [1] ✨ Cast Zoltraak (Ordinary Offensive Magic) [25 MP, 45-67 dm
    g, +50% vs Demon]");
117                 Console.WriteLine(" [2] 🛡️  Erect Hexagonal Defensive Barrier [15 MP, absorbs 75% ne
    xt attack]");
118                 Console.WriteLine($" [3] 🎭 Toggle Mana Suppression [Current: {(player.IsSuppressing
    Mana ? "Suppressed (Deceiving Demon)" : "Full Aura Unleashed")}]");
119                 Console.WriteLine(" [4] 🪄  Cast Basic Magic [10 MP, 20 dmg]");
120                 Console.WriteLine(" [5] 🦯  Staff Strike [0 MP, 10 dmg]");
121                 Console.WriteLine(" [6] 🧘  Meditate [Restores 30 MP]");
122                 Console.WriteLine(" [7] 🛡️  Defensive Block [Halves next turn damage]");
123                 Console.Write("Action > ");
124
125                 string? input = Console.ReadLine()?.Trim();
126                 Console.WriteLine();
127
128                 switch (input)
129                 {
130                     case "1":
131                         actionExecuted = player.CastZoltraak(boss);
132                         break;
133                     case "2":
```

```
                        actionExecuted = player.DeployDefensiveBarrier();
                        break;
                    case "3":
                        player.ToggleManaSuppression();
                        actionExecuted = true;
                        break;
                    case "4":
                        actionExecuted = player.CastBasicSpell(boss);
                        break;
                    case "5":
                        player.Attack(boss, 10);
                        actionExecuted = true;
                        break;
                    case "6":
                        player.Meditate(30);
                        actionExecuted = true;
                        break;
                    case "7":
                        player.Block();
                        actionExecuted = true;
                        break;
                    default:
                        Console.WriteLine("Invalid selection. Please choose an action from 1 to 7.");
                        break;
                }
            }

            if (!boss.IsAlive)
                break;

            Console.WriteLine();
            Console.WriteLine($"—— {boss.Name}'s Turn ——");

            // Demon AI logic
            ExecuteDemonTurn(boss, player);

            // Natural end-of-turn recovery
            boss.RegenerateMana(10);

            turn++;
            Console.WriteLine();
            Console.WriteLine("Press any key to continue to the next round...");
            Console.ReadKey(true);
            Console.WriteLine();
        }

        Console.WriteLine(new string('=', 65));
        if (player.IsAlive)
        {
            Console.WriteLine($"✨ VICTORY: {player.Name} has triumphed over {boss.Name}!");
        }
        else
        {
            Console.WriteLine($"💀 DEFEAT: {boss.Name} overwhelmed {player.Name}.");
        }
        Console.WriteLine(new string('=', 65));

```

```
        Console.WriteLine("\nPress any key to return to the main menu...");
        Console.ReadKey(true);
    }

    private static void ExecuteDemonTurn(Demon demon, Mage opponent)
    {
        bool underestimating = demon.UnderestimatesOpponent(opponent);

        // Demon priority decision tree
        if (demon.Mana >= 30)
        {
            demon.CastInnateMagic(opponent);
        }
        else if (demon.Mana >= 15)
        {
            demon.CastSpell(opponent);
        }
        else if (demon.Health <= demon.MaxHealth * 0.3)
        {
            Console.WriteLine($"🛡️ {demon.Name} falls back into a defensive stance!");
            demon.Block();
        }
        else
        {
            demon.Attack(opponent, 14);
        }
    }

    private static void RunAutomatedDemonstration()
    {
        Console.WriteLine();
        PrintBanner();
        Console.WriteLine("📜 Running Automated Combat Simulation:");
        Console.WriteLine("   Frieren the Slayer vs Aura the Guillotine");
        Console.WriteLine("   (Demonstrating Mana Suppression deception, Zoltraak demon bonus, and Defensive Barrier)");
        Console.WriteLine(new string('=', 65));

        var frieren = new Mage("Frieren", 120, 200);
        var aura = new Demon("Aura the Guillotine", 110, 160, "Auserlese (Scales of Obedience)", "Seven Sages of Destruction");

        // Frieren conceals her mana
        frieren.SetManaSuppression(true);

        Console.WriteLine($"\n[Setup] Frieren's true mana: {frieren.Mana}/{frieren.MaxMana} MP. Apparent mana: {frieren.PerceivedMana} MP.");
        Console.WriteLine($"[Setup] Aura perceives Frieren's mana as trivial. Underestimating: {aura.UnderestimatesOpponent(frieren)}.");

        int step = 1;

        // Round 1: Aura casts innate magic, Frieren uses defensive barrier
        Console.WriteLine($"\n── Round {step++}: Aura strikes, Frieren prepares defense ──");
        frieren.DeployDefensiveBarrier();
        aura.CastInnateMagic(frieren);

        // Round 2: Frieren reveals true mana, turning the battle!
```

```
        Console.WriteLine($"\n—— Round {step++}: The Reveal ——");
        Console.WriteLine("Frieren: \"Aura... the mana you are looking at is restricted.\"");
        frieren.SetManaSuppression(false);
        Console.WriteLine("Aura: \"Impossible... that is a deception spanning over a thousand years...!\"");

        // Round 3: Frieren casts demon-piercing Zoltraak
        Console.WriteLine($"\n—— Round {step++}: Zoltraak Demonic Execution ——");
        frieren.CastZoltraak(aura);

        // Round 4: Finishing blow
        if (aura.IsAlive)
        {
            Console.WriteLine($"\n—— Round {step++}: The Final Verdict ——");
            Console.WriteLine("Frieren: \"Aura... k**l yourself.\"");
            frieren.CastZoltraak(aura);
        }

        Console.WriteLine(new string('=', 65));
        Console.WriteLine($"Simulation concluded. Frieren status: {(frieren.IsAlive ? "Alive" : "Defeated")}, Aura status: {(aura.IsAlive ? "Alive" : "Defeated")}");
        Console.WriteLine(new string('=', 65));
    }

    private static void DisplayCombatantHUD(Character character, Mage? playerContext = null)
    {
        string hpBar = GenerateBar(character.Health, character.MaxHealth, 16);

        if (character is Mage mage)
        {
            string mpBar = GenerateBar(mage.Mana, mage.MaxMana, 12);
            string suppressionTag = mage.IsSuppressingMana ? " [🎭 Concealing Mana]" : " [🌊 Full Mana Aura]";
            string barrierTag = mage.HasDefensiveBarrier ? " [🛡️ Barrier Active]" : "";
            string blockTag = mage.IsBlocking ? " [🛡️ Blocking]" : "";

            Console.WriteLine($"🧙 {mage.Name,-12} HP: {hpBar} {mage.Health,3}/{mage.MaxHealth} | MP: {mpBar} {mage.Mana,3}/{mage.MaxMana}{suppressionTag}{barrierTag}{blockTag}");
        }
        else if (character is Demon demon)
        {
            string mpBar = GenerateBar(demon.Mana, demon.MaxMana, 12);
            string underestimationTag = (playerContext != null && demon.UnderestimatesOpponent(playerContext)) ? " [⚠️ Underestimating Opponent]" : "";
            string blockTag = demon.IsBlocking ? " [🛡️ Blocking]" : "";

            Console.WriteLine($"😈 {demon.Name,-12} HP: {hpBar} {demon.Health,3}/{demon.MaxHealth} | MP: {mpBar} {demon.Mana,3}/{demon.MaxMana}{underestimationTag}{blockTag}");
        }
        else
        {
            Console.WriteLine($"👤 {character.Name,-12} HP: {hpBar} {character.Health,3}/{character.MaxHealth}");
        }
    }

    private static string GenerateBar(int current, int max, int totalBlocks)
    {
        if (max <= 0) return new string('░', totalBlocks);
```

```
        int filled = Math.Clamp((int)Math.Round((double)current / max * totalBlocks), 0, totalBlocks);
        return $"[{new string('▓', filled)}{new string('░', totalBlocks - filled)}]";
    }

    private static void PrintBanner()
    {
        Console.WriteLine("╔══════════════════════════════════════════════════════════════╗");
        Console.WriteLine("║   ✦   F R I E R E N :   B E Y O N D   J O U R N E Y ' S   E N D ✦ ║");
        Console.WriteLine("║           Combat & Magic System — Built with .NET 10          ║");
        Console.WriteLine("╚══════════════════════════════════════════════════════════════╝");
        Console.WriteLine();
    }
}
```

11. Implemented Behavior vs. Pedagogical Extensions

To assist curriculum designers planning student assignments, problem sets, or laboratory exams, the table below delineates current production behavior from potential pedagogical extensions:

Area	Currently Implemented Behavior	Recommended Student Extensions
Class Hierarchy	Concrete `Character`, `Mage`, and `Demon` classes.	Add a `Warrior` subclass (e.g. Stark / Eisen) featuring physical stamina and weapon durability.
Spell Catalog	Four discrete actions: Basic Magic, Zoltraak, Barrier, and Meditate.	Refactor into a polymorphic `ISpell` interface with a `Grimoire` collection class.
Items & Inventory	No consumable items or inventory structures.	Implement healing potions, mana recovery elixirs, and equipable magical staves.
Battle Format	1-on-1 single combat duel.	Expand combat loop into party combat (Frieren, Fern, and Stark vs. Demon Trio).
Data Persistence	Transient in-memory execution; resets on exit.	Add JSON serialization to save and load character sheets and campaign states.
Unit Testing	Automated headless script via `--demo` flag.	Create an xUnit test suite testing damage formulas, barrier absorption, and suppression edge cases.